Автономная некоммерческая образовательная организация высшего образования

«Научно-технологический университет «Сириус»»

Лабораторная работа № 2

«Регистрация показаний аналоговых датчиков»

Вариант 2

По дисциплине: Основы программирования для микроэлектроники

Выполнил:

Студент

Мегерян Сергей Сергеевич

Проверил:

_____

ФТ Сириус, 2026 г.

Содержание

Тема

Цель работы

Оборудование

Схема подключения

Описание алгоритма

Основные формулы

- Среднее арифметическое
- Среднеквадратичное отклонение

Результат работы программы

Вывод

Тема

Дискретный сбор и статистическая обработка показаний аналогового датчика с выдачей данных в монитор порта.

Цель работы

Освоить считывание аналоговых сигналов с потенциометра, реализовать программный антидребезг кнопки, организовать хранение последних измерений в кольцевом буфере и вычислять основные статистические характеристики на скользящем интервале.

Оборудование

- Arduino Uno
- потенциометр
- кнопка
- макетная плата
- соединительные провода

Схема подключения

- Потенциометр: 5V - A0 - GND
- Кнопка: D2 - GND

Описание алгоритма

По нажатию кнопки контроллер считывает значение с аналогового входа A0. Перед фиксацией нажатия выполняется программный антидребезг с временной задержкой 50 мс. Каждое новое измерение сохраняется в кольцевом буфере размером 10 элементов. После обновления буфера программа пересчитывает среднее арифметическое и среднеквадратичное отклонение по всем сохраненным значениям. По команде пользователя через Serial Monitor результаты выводятся в текстовом виде.

Основные формулы

Среднее арифметическое

$$M = (x1 + x2 + ... + xn) / n$$

Среднеквадратичное отклонение

sigma = sqrt(((x1 - M)^2 + ... + (xn - M)^2) / n)

Результат работы программы

Система сохраняет не более десяти последних измерений. При переполнении буфера самое старое значение автоматически заменяется новым. Пользователь может в любой момент открыть монитор порта и запросить текущие данные командой d.

Вывод

В ходе работы был реализован дискретный сбор аналоговых данных с подавлением ложных срабатываний кнопки. Кольцевой буфер позволил организовать скользящее окно измерений, а расчет среднего и стандартного отклонения обеспечил базовую статистическую обработку сигнала.

Таблица соединений

Элемент	Подключение
Потенциометр	5V - A0 - GND
Кнопка измерения	D2 -> кнопка -> GND
Вывод результатов	Serial Monitor, 9600 бод

Иллюстрации и скриншоты

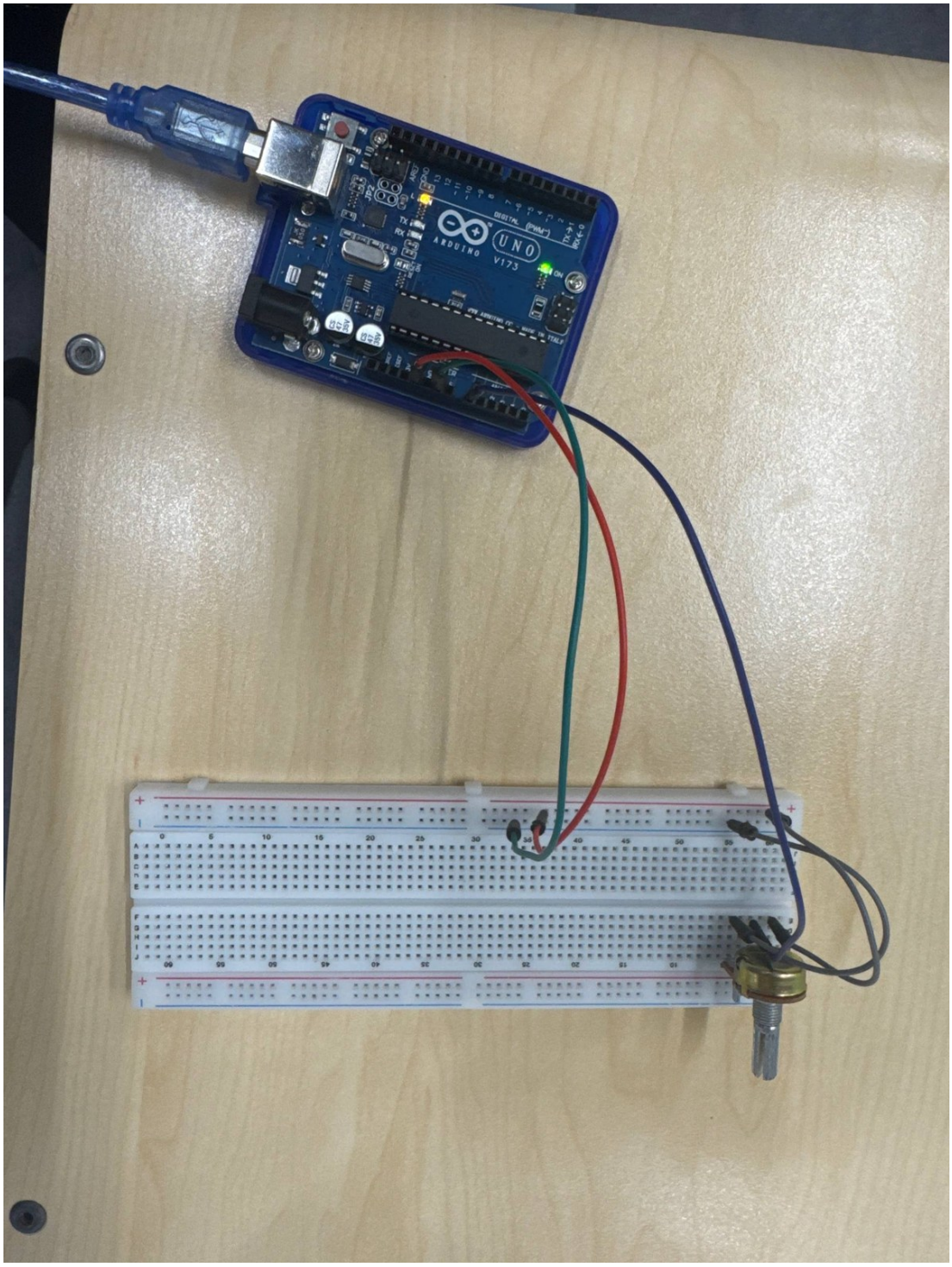

Рисунок 1 - Реальное подключение потенциометра и кнопки

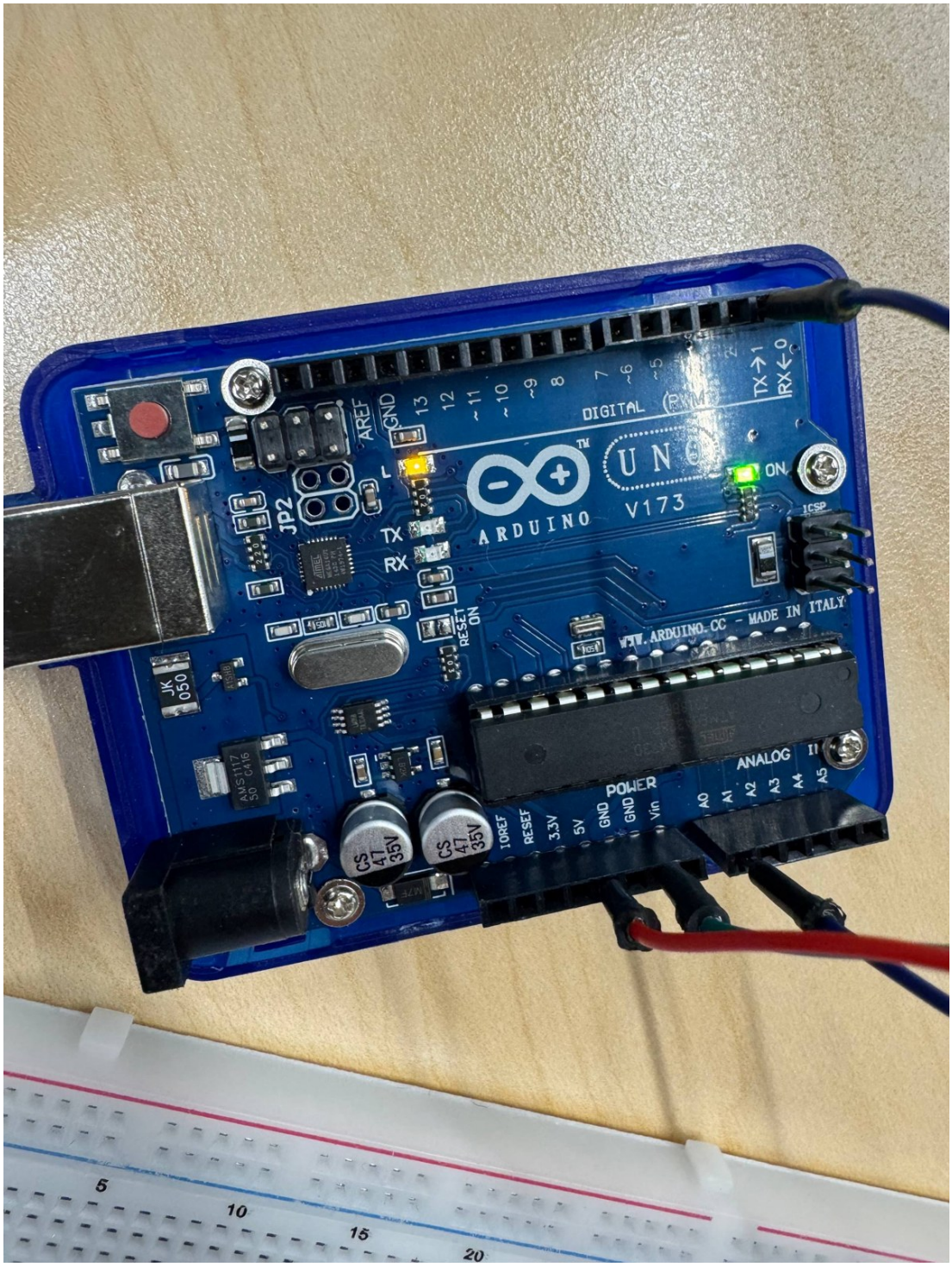

Рисунок 2 - Подключение потенциометра к Arduino Uno

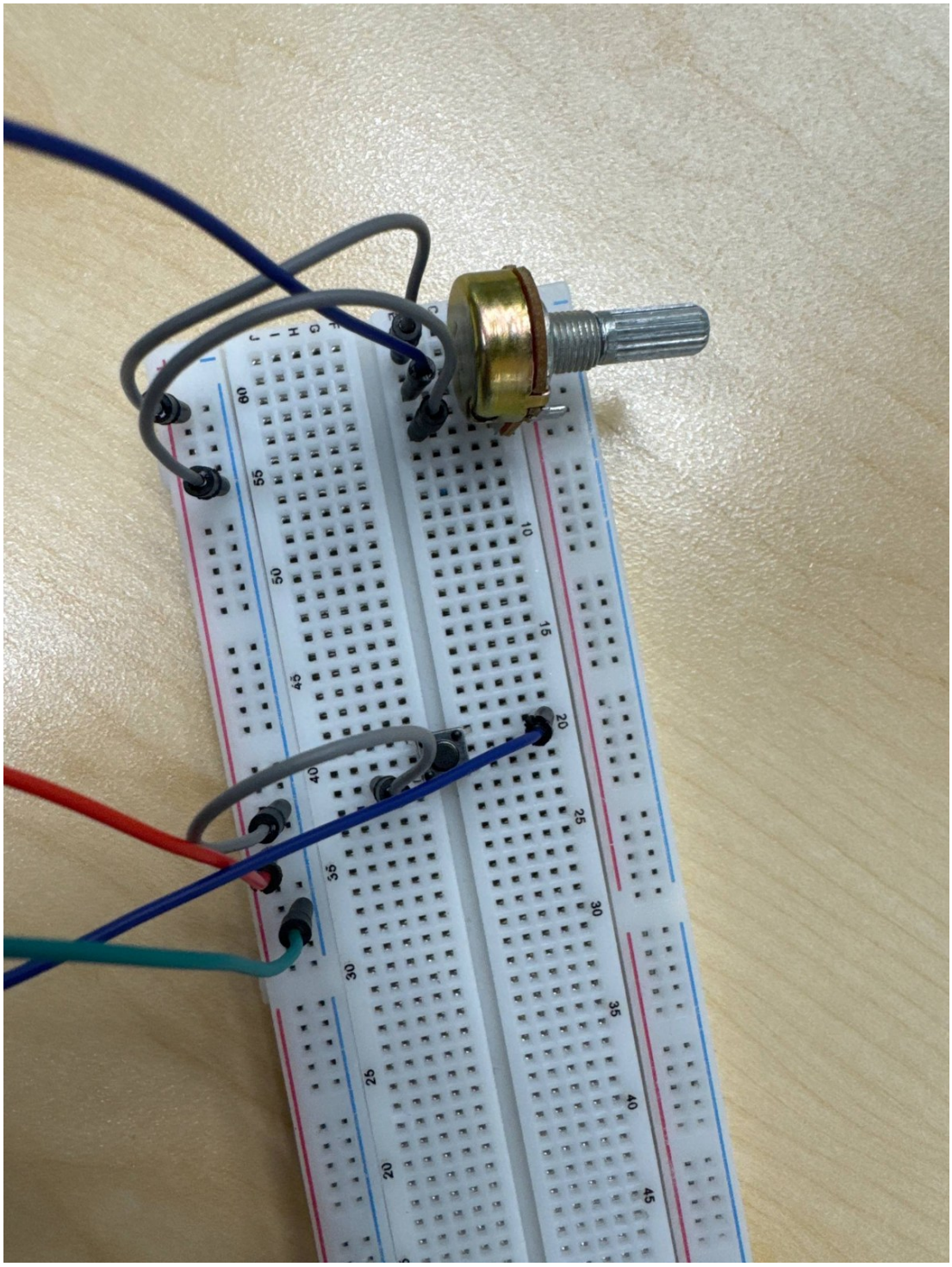

Рисунок 3 - Крупный план узла с потенциометром

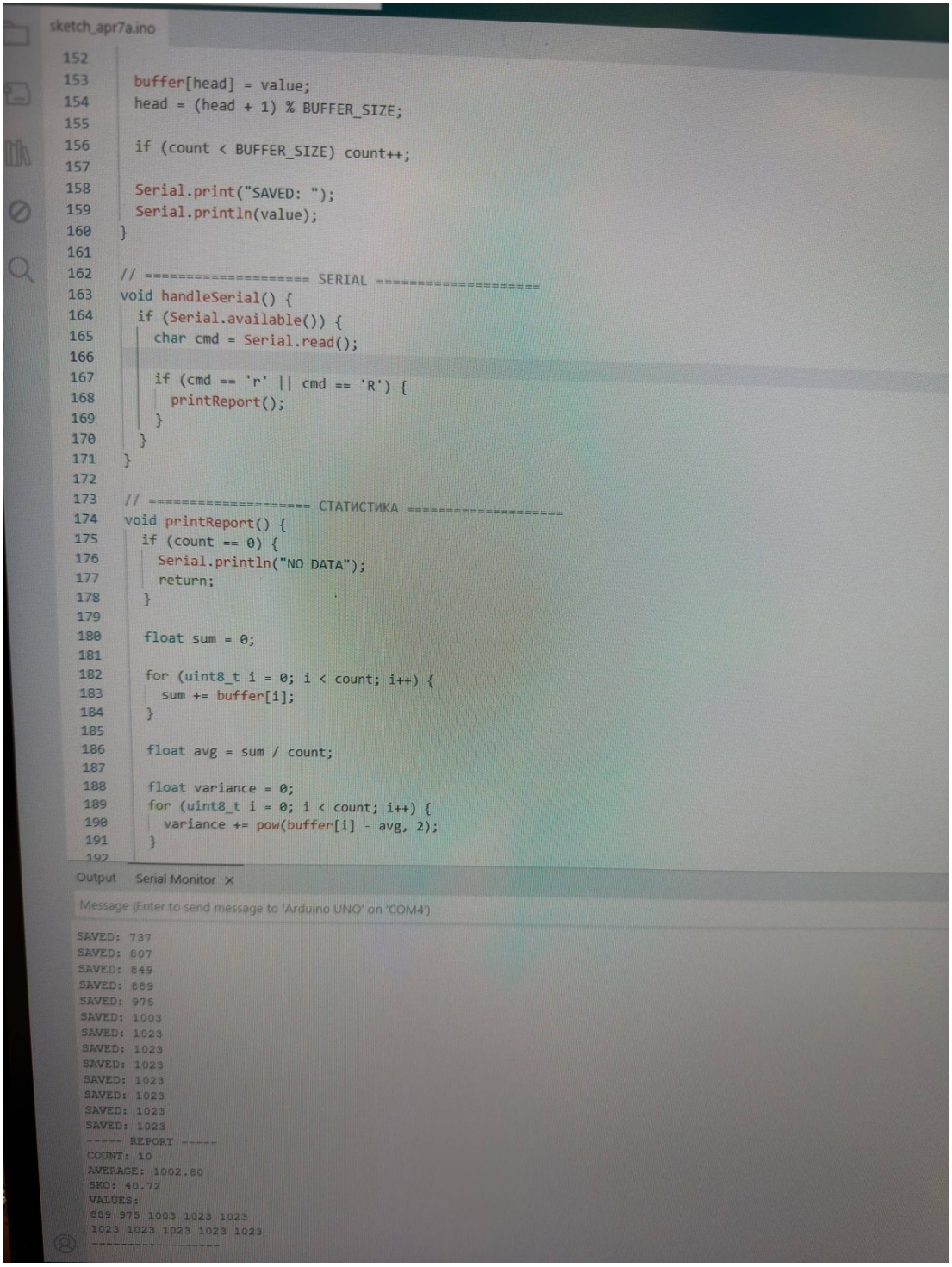

Рисунок 4 - Сохранение измерений в Serial Monitor

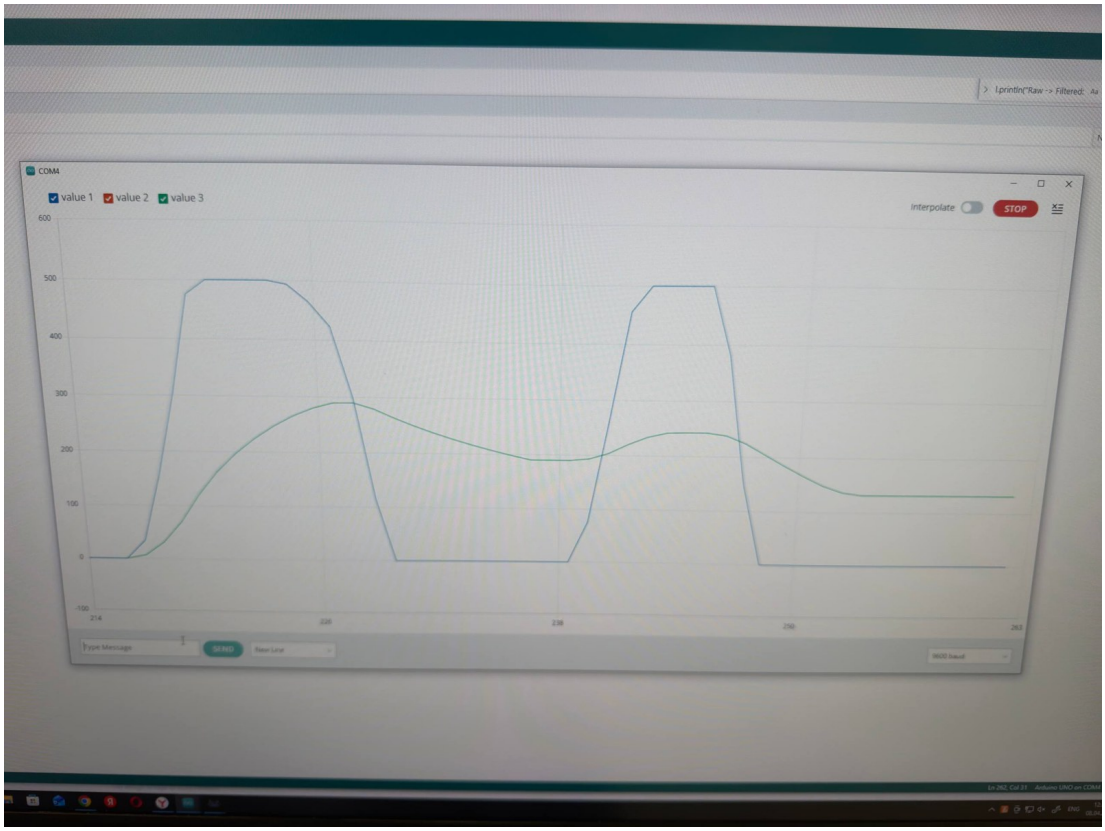

Рисунок 5 - Графическое представление результатов измерения

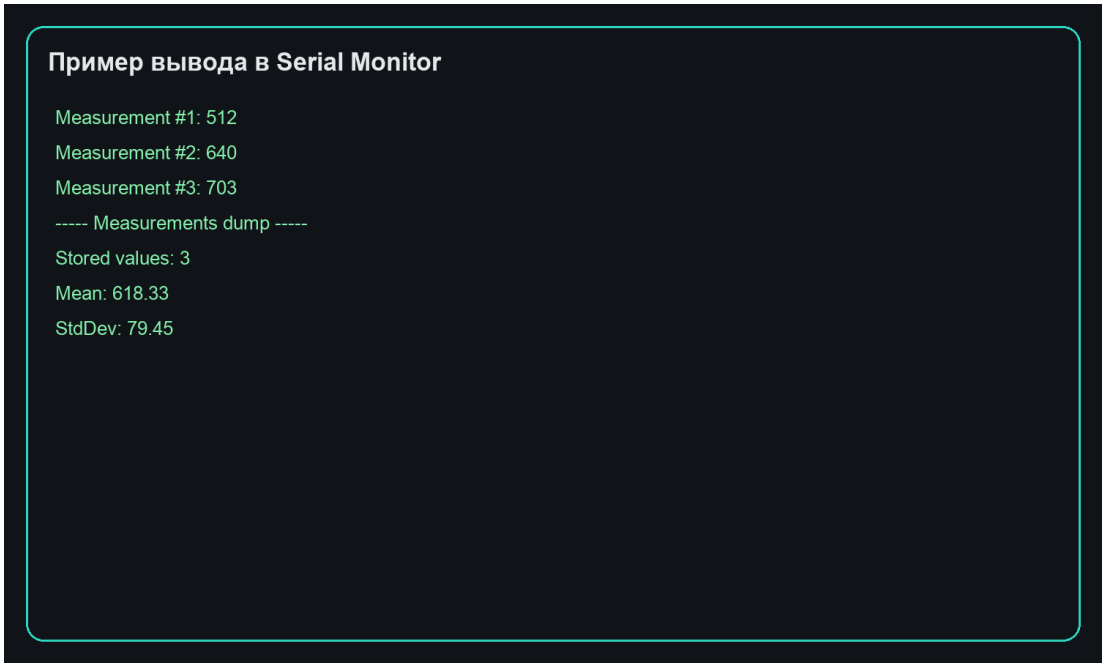

Рисунок 6 - Пример вычисления статистики

Приложение. Листинг программы

lab2_variant2.ino

```
#include <Arduino.h>
#include <math.h>


const uint8_t SENSOR_PIN = A0;
const uint8_t SAMPLE_BUTTON_PIN = 2;


const uint8_t BUFFER_SIZE = 10;
const unsigned long DEBOUNCE_DELAY_MS = 50;


uint16_t measurements[BUFFER_SIZE];
uint8_t bufferHead = 0;
uint8_t storedCount = 0;
unsigned long measurementCounter = 0;


bool lastButtonReading = HIGH;
bool stableButtonState = HIGH;
unsigned long lastDebounceTime = 0;


float currentAverage = 0.0f;
float currentStdDeviation = 0.0f;


void recalculateStatistics() {
  if (storedCount == 0) {
    currentAverage = 0.0f;
    currentStdDeviation = 0.0f;
    return;
  }


  float sum = 0.0f;
  for (uint8_t i = 0; i < storedCount; i++) {
    sum += measurements[i];
  }
  currentAverage = sum / storedCount;


  float varianceAccumulator = 0.0f;
  for (uint8_t i = 0; i < storedCount; i++) {
```

```
    float diff = measurements[i] - currentAverage;
    varianceAccumulator += diff * diff;
  }
  currentStdDeviation = sqrt(varianceAccumulator / storedCount);
}


void storeMeasurement(uint16_t value) {
  measurements[bufferHead] = value;
  bufferHead = (bufferHead + 1) % BUFFER_SIZE;


  if (storedCount < BUFFER_SIZE) {
    storedCount++;
  }


  measurementCounter++;
  recalculateStatistics();
}


uint8_t oldestIndex() {
  if (storedCount < BUFFER_SIZE) {
    return 0;
  }


  return bufferHead;
}


void printMeasurements() {
  Serial.println(F("----- Measurements dump -----"));
  Serial.print(F("Stored values: "));
  Serial.println(storedCount);
  Serial.print(F("Mean: "));
  Serial.println(currentAverage, 2);
  Serial.print(F("StdDev: "));
  Serial.println(currentStdDeviation, 2);


  if (storedCount == 0) {
    Serial.println(F("Buffer is empty."));
    Serial.println(F("-----------------------------"));
    return;
  }
```

```
  uint8_t start = oldestIndex();
  for (uint8_t i = 0; i < storedCount; i++) {
    uint8_t idx = (start + i) % BUFFER_SIZE;
    Serial.print(i + 1);
    Serial.print(F(": "));
    Serial.println(measurements[idx]);
  }

  Serial.println(F("-----------------------------"));
}

void printHelp() {
  Serial.println(F("Commands:"));
  Serial.println(F("d - dump measurements and statistics"));
  Serial.println(F("h - print help"));
}

void handleSerialCommands() {
  while (Serial.available() > 0) {
    char command = static_cast<char>(Serial.read());

    if (command == 'd' || command == 'D') {
      printMeasurements();
    } else if (command == 'h' || command == 'H') {
      printHelp();
    }
  }
}

void handleSampleButton() {
  bool reading = digitalRead(SAMPLE_BUTTON_PIN);

  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
    lastButtonReading = reading;
  }

  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY_MS && reading !=
stableButtonState) {
    stableButtonState = reading;
```

```
    if (stableButtonState == LOW) {
      uint16_t measurement = analogRead(SENSOR_PIN);
      storeMeasurement(measurement);


      Serial.print(F("Measurement #"));
      Serial.print(measurementCounter);
      Serial.print(F(": "));
      Serial.println(measurement);
    }
  }
}


void setup() {
  pinMode(SAMPLE_BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(9600);


  Serial.println(F("Lab 2 / Variant 2 ready."));
  printHelp();
}


void loop() {
  handleSampleButton();
  handleSerialCommands();
}
```